

Molecular Docking Workflow Using Gaussian

Step 1: Preparing the SMILES Data

We begin with molecular data in the SMILES (Simplified Molecular Input Line Entry System) format, which provides a textual representation of chemical structures. This data is extracted from a spreadsheet and saved into a file named `'all.txt'`, with each SMILES string written on a single line. Consistency in this format is critical, as the subsequent steps rely on structured input to function correctly.

Step 2: Converting SMILES to MOL2 Format

The next step is to convert the SMILES strings into MOL2 format, a 3D representation of molecular structures. This is achieved using the `'convert_smiles_mol2_gauss.py'` script. Here's what this program does:

1. Cleans the Input Data: The program removes invalid characters, such as wildcards (*), from the SMILES strings to ensure they are compatible with downstream tools.
2. Uses Open Babel: It employs Open Babel to add hydrogen atoms, generate 3D coordinates, and output the molecules in MOL2 format.
3. Logs Errors: Any issues during the conversion process are recorded in an error log for troubleshooting.

This step ensures that the molecules are represented in 3D space, making them suitable for advanced computational analysis.

Step 3: Generating Gaussian Input File

With the MOL2 file prepared, we use the `'ConvertToGauss_v2.py'` script to generate Gaussian Input Files (`'gjf'`). This program:

1. Splits the MOL2 File: It separates the MOL2 file into individual molecular structures.
2. Adds Gaussian Headers: Each Gaussian input file is prefixed with standard Gaussian headers, including computational methods, charge, and multiplicity.
3. Outputs to a Folder: All `'gjf'` files are stored in a directory named `'gaussian_inputs'`.

These `'gjf'` files are formatted to comply with Gaussian's requirements, enabling the computation of molecular properties and optimized geometries.

Step 4: Transferring Files to the Server

Once the `'gjf'` files are ready, we transfer the `'gaussian_inputs'` folder to the server using WinSCP, a secure file transfer tool. This step ensures that the input files are available on the server where Gaussian is installed, enabling us to run the calculations.

Step 5: Executing Gaussian Jobs and Transferring Outputs

On the server, we prepare a list of Gaussian input files using the following command:

```
ls /home/hrajp2/test_coconut_200/gaussian_inputs/*.gjf > /home/hrajp2/test_coconut_200/file_list.txt
```

This file list is used by the `'mpi_gaussian.py'` program in conjunction with the SLURM job scheduler script `'mpi_job_gauss.slurm'`. Here's what happens:

1. **Parallel Execution:** The SLURM script enables MPI-based Gaussian execution, which distributes the workload across multiple compute nodes, significantly speeding up the calculations.
2. **Gaussian Computations:** Each input file is processed, generating `'.out'` and `'.log'` files that contain critical data, such as optimized molecular geometries and energy calculations.

After the computations are complete, we copy the `'output_gaussian'` folder back to the local machine using WinSCP. This allows us to proceed with post-processing locally.

Step 6: Extracting Molecular Orientations

With the Gaussian outputs on our local machine, we use the `'program_stan_input_orientation.py'` script to extract the Input Orientation or Standard Orientation data from the log files. This program:

1. **Identifies Relevant Sections:** It scans the log files for sections containing atomic coordinates.
2. **Formats Output:** The extracted data is saved as `'.txt'` files for further processing.

This ensures that we retain the optimized molecular geometries calculated during the Gaussian jobs.

Step 7: Converting to XYZ Format

We then use the `'convert_to_xyz_2'` program to transform the extracted orientations into the XYZ format, a simple and widely accepted 3D molecular structure representation. These XYZ files are then transferred back to the server for further conversion.

Step 8: Converting XYZ to SDF Format

On the server, we run the `'xyz_to_sdf_linux.py'` script to convert the XYZ files into SDF (Structure Data File) format. The SDF format is widely used in cheminformatics and is essential for docking simulations. Running this script on the server ensures computational compatibility and accuracy.

Step 9: Preparing Ligands for Docking

The ligands are prepared for docking using the following steps:

1. **Energy Minimization:** We optimize the ligand geometries with the command:

```
obminimize -ff MMFF94 -n 1000 *.sdf
```

2. **Conversion to PDBQT:** The minimized ligands are converted to the PDBQT format using:

```
obabel -isdf *.sdf -opdbqt -O*.pdbqt
```

3. Cleanup: Unnecessary files are deleted, and a 'ligand.txt' file listing all ligand names is created.

This preparation step ensures that the ligands are ready for accurate docking simulations.

Step 10: Docking Simulations

The docking simulations are conducted using:

- Receptor File ('receptor.pdbqt'): Contains the protein structure.
- Configuration File ('conf.txt'): Defines the docking grid box and parameters.
- MPI Docking Scripts: The 'mpi4.py' program and its associated SLURM script execute the docking simulations in parallel.
- Mpi slurm file

The docking results, including binding scores, are summarized using:

```
tail -n11 *.log > results.txt
```

This extracts the key docking metrics for analysis.

Step 11: Final Analysis

The final step is to process the docking results using the 'result.py' program, which generates a CSV file containing detailed docking scores and other critical data. This file serves as the final output of the workflow, ready for interpretation and further study.

Commands Step 10, You can find this on youtube:

----INSTALL UBUNTU-----

UPGRADE AND UPDATE LIBRARIES OF UBUNTU BY FOLLOWING COMMANDS

Commands:

sudo apt update

sudo apt upgrade

sudo apt install gcc

sudo apt install cmake

sudo apt install build-essential

sudo apt install libfftw3-dev OR

sudo apt-get install -y libfftw3-dev

install Openbabel

sudo apt install openbabel

install Vinna

sudo apt install autodock-vina

#-----#

#STEPS FOR MULTIPLE LIGANDS DOCKING

First you need to perform docking in UCSF-Chimera, with any one ligand, once this is done, you can use the output "docking.receptor.pdbqt" file for following steps.

#Place the docking.receptor.pdbqt or file having the name like "xxxx.receptor.pdbqt" in to a new folder (working directory), note: this file will be generated previously while performing docking in UCSF-Chimera.

rename this file as simply "receptor.pdbqt"

place the files "conf.txt" and script "Vina.sh" in the working directory

At this point you will have following files in the working directory: (1) receptor.pdbqt, (2)conf.txt, (3)two scripts and (4) ligand multiple files or single file with multiple ligands in sdf format.

open and edit "conf.txt" to enter the position coordinates for X,Y and Z previously noted down while making docking grid in autodock vina and also enter the size of the grid box.

If you have a single sdf file with multiple ligands here and run following command, and if you have multiple ligands separately in sdf format in working folder skip this step,

obabel -isdf nihcc-in-man.sdf-osdf -O *.sdf --split

DRUG DISCOVERY | GAUSSIAN

Minimization of ligands

obminimize

obminimize -ff MMFF94 -n 1000 *.sdf

#convert ligands in sdf to pdbqt format

obabel -isdf *.sdf -opdbqt -O*.pdbqt

#Once this step is done, delete the sdf files of ligands from the working folder.

Make ligand text file for info.

ls > ligand.txt

Open ligand.txt and delete all other lines other than ligand names.

Run the script for docking, here you should have "receptor.pdbqt" file or anywhere which is optimized and minimized.

ligand.txt #ask you for text file

To prepare result table

tail -n11 *.log>results.txt

Save the appropriate pose using File > Save PDB > input file name, choose protein and pose of choice by pressing ctrl select> choose "a single file" under "Save multiple models in" and lastly press Save.

Scripts:



Docking_Gaussian_Programs_Hemant_Rajpal.rar